

Feature EKF Slam Localization

Pravin Oli, Gebreslassie Gebrecherkos Gebrezgabhier

Hands-On Localization, IFROS

pravin.oli.08@gmail.com, cherqos.gebre@gmail.com

March 16, 2025

Abstract

This lab explores Feature-based Extended Kalman Filter Simultaneous Localization and Mapping (FEKF-SLAM) for a differential drive mobile robot. The lab is divided into three main parts: (1) Localization without feature measurements (2) SLAM with a priori-known features and (3) Full SLAM. The results demonstrate the effectiveness of feature-based corrections in improving localization accuracy, while challenges related to data association and state estimation were addressed.

1 Introduction

This lab focuses on **Feature-based Extended Kalman Filter Simultaneous Localization and Mapping (FEKF-SLAM)**, where a **differential drive mobile robot** localizes itself while incrementally constructing a map of its surroundings.

The lab is divided into three main parts:

- **Localization without feature measurements** – The robot estimates its motion using only odometry data, implementing the prediction step of the **Extended Kalman Filter (EKF)**.
- **SLAM with a priori-known features** – The robot localizes itself using a known set of features and updates its state using **measurement updates**.
- **Full SLAM** – The robot dynamically detects and adds new features to its map, performing simultaneous localization and mapping.

2 Mathematical and Theoretical Background

The Feature-based Extended Kalman Filter (FEKF-SLAM) provides a probabilistic framework for solving this problem by maintaining a state vector containing both the robot's pose and the positions of environmental features.

2.1 Spatial Representation and State Vector

A map is represented as a stochastic state vector:

$$\mathbf{x}_k = \begin{bmatrix} {}^N\mathbf{x}_B^T & {}^N\mathbf{x}_{F_1}^T & {}^N\mathbf{x}_{F_2}^T & \dots & {}^N\mathbf{x}_{F_n}^T \end{bmatrix}^T \quad (1)$$

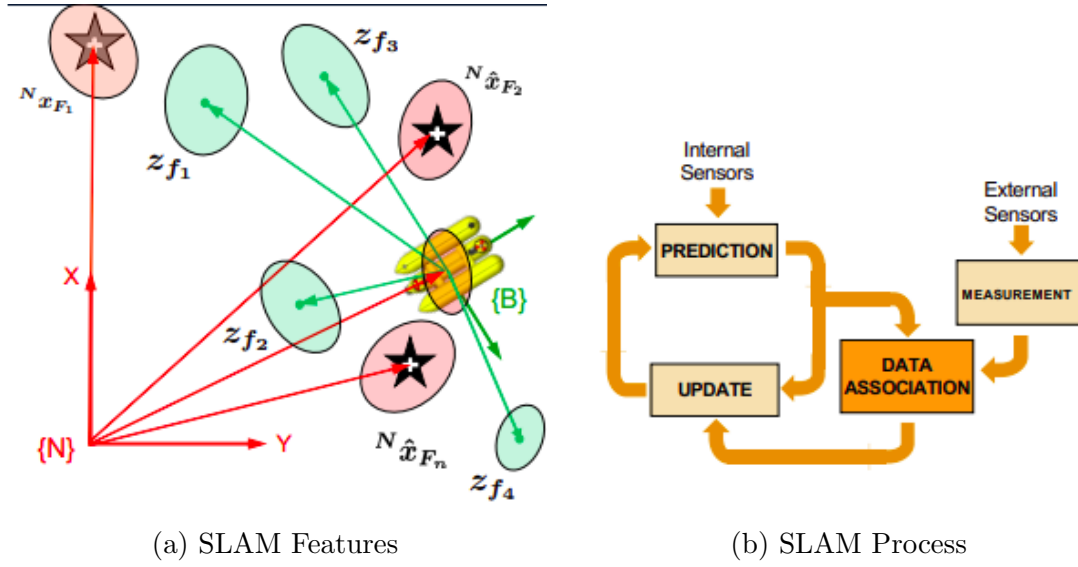


Table 1: Images Adopted from Lecture Slides

where:

- ${}^N\mathbf{x}_B$ represents the robot's pose in the world frame.
- ${}^N\mathbf{x}_{F_i}$ represents the feature locations in the world frame.

The covariance matrix associated with the state vector is given by:

$${}^N\mathbf{P}_k = \begin{bmatrix} {}^NP_{B,B} & {}^NP_{B,F_1} & {}^NP_{B,F_2} & \dots & {}^NP_{B,F_n} \\ {}^NP_{F_1,B} & {}^NP_{F_1,F_1} & {}^NP_{F_1,F_2} & \dots & {}^NP_{F_1,F_n} \\ {}^NP_{F_2,B} & {}^NP_{F_2,F_1} & {}^NP_{F_2,F_2} & \dots & {}^NP_{F_2,F_n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ {}^NP_{F_n,B} & {}^NP_{F_n,F_1} & {}^NP_{F_n,F_2} & \dots & {}^NP_{F_n,F_n} \end{bmatrix} \quad (2)$$

This matrix encodes:

- The robot's pose uncertainty ${}^NP_{B,B}$.
- The feature uncertainties ${}^NP_{F_i,F_i}$.
- The correlation between the robot and features ${}^NP_{B,F_i}$.

2.2 SLAM Algorithm

Algorithm 1 Method LocalizationLoop($\hat{x}_0, P_0$)

- 1: $(\hat{x}_{k-1}, P_{k-1}) \leftarrow (\hat{x}_0, P_0)$
 - 2: $[z_f, R_f, H_f, V_f] \leftarrow \text{GetFeatures}()$
 - 3: $[\hat{x}_k, P_k] \leftarrow \text{AddNewFeatures}({}^N\hat{x}_{k-1}, {}^NP_{k-1}, z_f, R_f)$
 - 4: **for** $k = 1$ to ∞ **do**
 - 5: $[\hat{x}_k, P_k] \leftarrow \text{Localize}(\hat{x}_{k-1}, P_{k-1})$
 - 6: **end for**
-

Algorithm 2 Method Localize($\hat{x}_{k-1}, u_k$)

```

1:  $[u_k, Q_k] \leftarrow \text{GetInput}()$ 
2:  $[\hat{x}_k, P_k] \leftarrow \text{Prediction}(\hat{x}_{k-1}, P_{k-1}, u_k, Q_k)$ 
3:  $[z_m, R_m, H_m, V_m] \leftarrow \text{GetMeasurements}()$ 
4:  $[z_f, R_f, H_f, V_f] \leftarrow \text{GetFeatures}()$ 
5:  $H_p \leftarrow \text{DataAssociation}(\hat{x}_k, P_k, z_f, R_f)$ 
6:  $[z_k, R_k, H_k, V_k, z_{np}, R_{np}] \leftarrow \text{StackMeasurementsAndFeatures}(z_m, R_m, H_m, V_m, z_f, R_f, H_p)$ 
7:  $[\hat{x}_k, P_k] \leftarrow \text{Update}(\hat{x}_k, P_k, z_k, H_k, V_k)$ 
8:  $[^N\hat{x}_k, ^N P_k] \leftarrow \text{AddNewFeatures}(^N\hat{x}_k, ^N P_k, z_{np}, R_{np})$ 
9: return  $(\hat{x}_k, P_k)$ 

```

2.3 Localization and Mapping Equations

2.3.1 Prediction Step

The state prediction follows the motion model:

$$\mathbf{x}_{k|k-1} = f(\mathbf{x}_{k-1}, \mathbf{u}_k) + \mathbf{w}_k \quad (3)$$

where:

- $f(\mathbf{x}, \mathbf{u})$ represents the motion model.
- $\mathbf{u}_k$ is the control input.
- $\mathbf{w}_k$ is the process noise.

The covariance prediction is given by:

$$\mathbf{P}_{k|k-1} = \mathbf{F}_k \mathbf{P}_{k-1} \mathbf{F}_k^T + \mathbf{Q}_k \quad (4)$$

where:

- $\mathbf{F}_k = \frac{\partial f}{\partial \mathbf{x}}$ is the Jacobian of the motion model.
- $\mathbf{Q}_k$ is the process noise covariance.

2.3.2 Measurement Update Step

When a feature is observed, the measurement model is:

$$\mathbf{z}_k = h(\mathbf{x}_k, \mathbf{v}_k) \quad (5)$$

where:

- $h(\mathbf{x})$ is the observation function.
- $\mathbf{v}_k$ is the measurement noise.

The innovation is computed as:

$$\mathbf{y}_k = \mathbf{z}_k - h(\mathbf{x}_{k|k-1}) \quad (6)$$

The Kalman gain is:

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}_k^T (\mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^T + \mathbf{R}_k)^{-1} \quad (7)$$

The state is updated as:

$$\mathbf{x}_k = \mathbf{x}_{k|k-1} + \mathbf{K}_k \mathbf{y}_k \quad (8)$$

And the covariance is updated as:

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_{k|k-1} \quad (9)$$

2.4 Feature Initialization and State Augmentation

When a new feature is observed, it is added to the state vector:

$$\mathbf{x}_k^+ = \begin{bmatrix} \mathbf{x}_k \\ \mathbf{x}_F \end{bmatrix}, \quad \mathbf{P}_k^+ = \begin{bmatrix} \mathbf{P}_k & \mathbf{P}_{xF} \\ \mathbf{P}_{Fx} & \mathbf{P}_F \end{bmatrix} \quad (10)$$

where:

- $\mathbf{x}_F$ represents the new feature position.
- $\mathbf{P}_F$ is its associated uncertainty.

The feature position is estimated using the inverse observation model:

$$\mathbf{x}_F = g(\mathbf{x}_k, \mathbf{z}_F) \quad (11)$$

Jacobian matrices for uncertainty propagation:

$$\mathbf{J}_g = \frac{\partial g}{\partial \mathbf{x}}, \quad \mathbf{J}_z = \frac{\partial g}{\partial \mathbf{z}} \quad (12)$$

2.5 Data Association and Feature Matching

To correctly associate observed features with mapped landmarks, we use Mahalanobis distance:

$$D^2 = (\mathbf{z}_i - \mathbf{h}_j)^T \mathbf{S}^{-1} (\mathbf{z}_i - \mathbf{h}_j) \quad (13)$$

where D^2 is the squared Mahalanobis distance, and $\mathbf{S}$ is the innovation covariance.

3 Methodology

This section describes the implementation of Feature-based Extended Kalman Filter Simultaneous Localization and Mapping (FEKF-SLAM). The methodology consists of three key parts:

3.1 Part I: Localization Without Feature Measurements (Prediction Step)

Description: The robot estimates its motion using the prediction step of the Extended Kalman Filter (EKF) without incorporating feature measurements.

Code Snippet:

Listing 1: Prediction Step

```
def Prediction(self, uk, Qk, xk_1, Pk_1):
    """
    Implements the prediction step of the FEKFSLAM algorithm.

    :param uk: Control input
    :param Qk: Covariance of motion model noise
    :param xk_1: State vector mean at time step k-1
    :param Pk_1: Covariance matrix at time step k-1
    :return: Predicted state vector mean (xk_bar) and covariance (
            Pk_bar)
    """

    self.xk_1 = xk_1 if xk_1 is not None else self.xk_1
    self.Pk_1 = Pk_1 if Pk_1 is not None else self.Pk_1

    # Compute state prediction using motion model
    xk_bar = xk_1.copy()
    xk_bar[0:3] = self.f(xk_1[0:3], uk)

    # Compute Jacobians
    Ak = self.Jfx(xk_1[0:3]) # Jacobian of motion model w.r.t. state
    Wk = self.Jfw(xk_1[0:3]) # Jacobian of motion model w.r.t. noise

    # Compute covariance prediction
    Pk_bar = Ak @ Pk_1 @ Ak.T + Wk @ Qk @ Wk.T

    return xk_bar, Pk_bar
```

3.2 Part II: SLAM with A Priori-Known Features (Correction Step)

Description: The robot incorporates known features into its state estimation using the measurement update step of the EKF.

Code Snippet:

Listing 2: Measurement Update

```
def Update(self, zk, Rk, xk_bar, Pk_bar, Hk, Vk):
    """
    Implements the measurement update step of the Extended Kalman
    Filter (EKF).

    :param zk: Observation vector
    :param Rk: Covariance matrix of observation noise
    :param xk_bar: Predicted state vector
    :param Pk_bar: Predicted covariance matrix
    :param Hk: Observation model Jacobian
    :param Vk: Measurement noise Jacobian
    :return: Updated state vector (xk) and covariance matrix (Pk)
    """
```

```

# Compute Kalman Gain
S_k = Hk @ Pk_bar @ Hk.T + Vk @ Rk @ Vk.T
Kk = Pk_bar @ Hk.T @ np.linalg.inv(S_k)

# Compute innovation
yk = zk - Hk @ xk_bar

# Update state estimate
xk = xk_bar + Kk @ yk

# Update covariance estimate
I = np.eye(len(xk_bar))
Pk = (I - Kk @ Hk) @ Pk_bar

return xk, Pk

```

3.3 Part III: Full SLAM (Adding New Features Dynamically)

Description: The robot detects previously unmapped features and adds them to its state vector.

Code Snippet:

Listing 3: Feature Initialization

```

def AddNewFeatures(self, xk, Pk, znp, Rnp):
    """
    Adds newly detected features to the SLAM state vector.

    :param xk: State vector mean
    :param Pk: State covariance matrix
    :param znp: Vector of newly observed, unmapped features
    :param Rnp: Covariance of new feature observations
    :return: Updated state vector (xk_plus) and covariance (Pk_plus)
    """

    xk_plus = xk.copy()
    Pk_plus = Pk.copy()

    for i in range(len(znp)):
        # Compute feature position using inverse observation model
        NxFi = self.g(xk[0:3], znp[i])
        xk_plus = np.vstack((xk_plus, NxFi))

        # Compute feature covariance
        Jgxi = self.Jgx(xk[0:3], znp[i])
        Jgvi = self.Jgv(xk[0:3], znp[i])
        NpFi = Jgxi @ Pk[0:3, 0:3] @ Jgxi.T + Jgvi @ Rnp[i] @ Jgvi.T

        # Expand covariance matrix
        Pk_plus = np.block([
            [Pk, Jgxi @ Pk[0:3, :]],
            [Pk[:, 0:3] @ Jgxi.T, NpFi]
        ])

    return xk_plus, Pk_plus

```

3.4 Feature-Based EKF-SLAM: Feature Observations

Description: The ‘FEKFSLAMFeature.py’ script extends the ‘MapFeature’ class to implement feature-based observation models for SLAM. Unlike traditional localization, where features are pre-mapped, this implementation allows mapping previously unknown features. The primary goal is to model feature observations as part of the state vector and compute their expected measurements.

Code Snippet:

Listing 4: Feature Observation Model

```
class FEKFSLAMFeature(MapFeature):
    """
    Extends the MapFeature class to implement Feature-based EKF-SLAM.
    Unlike traditional localization, this implementation maintains both
    the robot's pose and feature locations within the state vector.
    """

    def hfj(self, xk_bar, Fj):
        """
        Computes the expected observation of a feature given the
        current robot state.

        :param xk_bar: Mean of the predicted state vector
        :param Fj: Index of the observed feature in the map
        :return: Expected feature observation hfj
        """
        Nx_B = xk_bar[0:3,0].reshape((3,1)) # Extract robot pose
        Fj = CartesianFeature(xk_bar[self.xB_dim + Fj*2:self.xB_dim + (
            Fj+1)*2])
        hfj = self.s2o(Fj.boxplus(Pose3D.ominus(Nx_B)))
        return hfj

    def Jhfjx(self, xk, Fj):
        """
        Computes the Jacobian of the feature observation model with
        respect to the state.

        :param xk: State vector mean
        :param Fj: Index of the observed feature
        :return: Jacobian matrix Jhfjx
        """
        xB_dim = self.xB_dim
        Nx_B = xk[0:3, 0].reshape((3, 1))
        Fj_obj = CartesianFeature(xk[self.xBpose_dim + Fj * 2:self.
            xBpose_dim + (Fj + 1) * 2])

        Js2o = self.J_s2o(Fj_obj.boxplus(Pose3D.ominus(Nx_B)))
        J_1boxplus = Fj_obj.J_1boxplus(Pose3D.ominus(Nx_B))
        J_ominus = Pose3D.J_ominus(Nx_B)
        J2_boxplus = Fj_obj.J_2boxplus(Pose3D.ominus(Nx_B))

        J = np.zeros((2, xk.shape[0]))
        J[:2, :3] = Js2o @ J_1boxplus @ J_ominus
        J[:2, 3 + 2 * Fj: 3 + 2 * (Fj + 1)] = Js2o @ J2_boxplus

        return J
```

4 Results and Discussion

This section presents the results obtained from the implementation of Feature-based Extended Kalman Filter Simultaneous Localization and Mapping (FEKF-SLAM). The results are divided into three parts:

4.1 Part I: Localization Without Feature Measurements

Result: The robot’s estimated trajectory follows the motion model but exhibits increasing uncertainty over time due to the absence of feature-based corrections. As shown in Figure 1, the covariance ellipses (blue) grow, indicating drift accumulation in the state estimation.

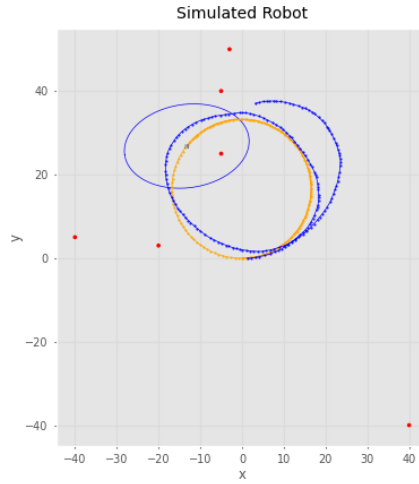


Figure 1: Localization without feature Measurements. Part I.

Discussion: Without feature updates, the uncertainty increases due to error propagation in odometry-based localization. This demonstrates the need for sensor-based corrections in SLAM.

4.2 Part II: SLAM with A Priori-Known Features

Result: The inclusion of known features (red points) enables state corrections. As shown in Figure 2, the robot updates its position by incorporating feature measurements, significantly reducing uncertainty.

Discussion: The results confirm that feature-based corrections significantly reduce localization errors. The Kalman filter update step adjusts the trajectory based on observed landmarks, ensuring more accurate state estimation.

4.3 Part III: Full SLAM with Dynamic Feature Mapping

Result: The system successfully detects and integrates previously unmapped features into the SLAM state vector. As shown in Figure, new features are added dynamically, improving trajectory estimation.

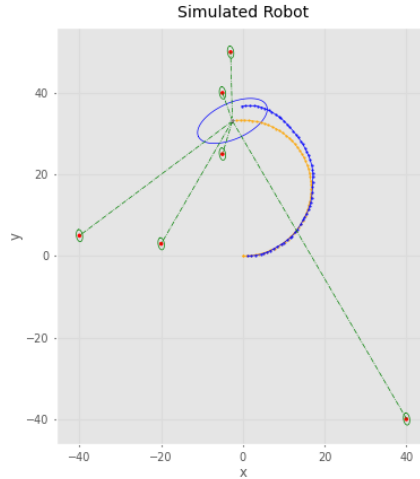
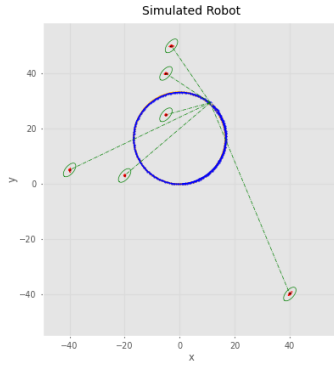
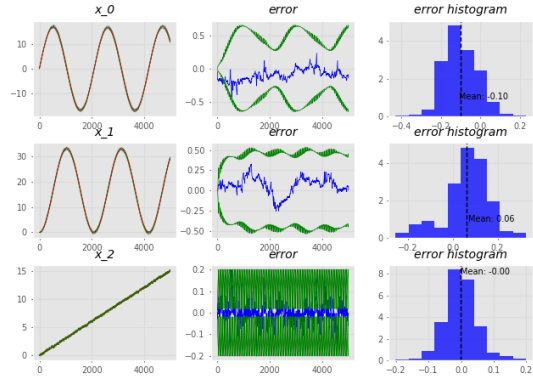


Figure 2: SLAM with Priori-Known Features Part II.



(a) SLAM Feature Mapping.



(b) Error

Table 2: Full SLAM with Dynamic Feature Mapping. Part III

5 Problems Encountered

During the implementation of FEKF-SLAM, several challenges were encountered, affecting state prediction, update steps, and feature association. One major issue was the lack of valid feature observations. The problem were cause due to:

- Inconsistencies in state prediction and update steps.
- Incorrect feature initialization, covariance expansion errors, and high uncertainty in new feature positions

6 Conclusion

In this lab we successfully implemented Feature-based Extended Kalman Filter Simultaneous Localization and Mapping (FEKF-SLAM) for a differential drive robot. The results demonstrated the impact of feature-based corrections in reducing localization drift and

improving state estimation. The new features were efficiently integrated, enabling the construction of a real-time map. However, challenges such as data association errors, prediction inconsistencies, and feature initialization issues were encountered and addressed. In conclusion, The FEKF SLAM reduces the error and over drift of dead reckoning by continuously updating new features and robot state.